

4 (a) Bagging-based axis-orthogonal random forest

Stage	Key operations
1. Bootstrap aggregation (bagging)	- For each of T trees draw, with replacement, a sample of size <i>N</i> from the original training set (<i>N</i>≈ size of original set). - Instances not drawn form the out-of-bag (OOB) subset for that tree and are later used for unbiased error & feature-importance estimates.
2. Growing a single tree	Axis-orthogonal means each internal split tests one variable, one threshold. At a node containing <i>n</i> samples: 1 Feature sub-sampling - randomly choose <i>m_try</i> features out of <i>p</i> (typical defaults: √p for classification, p/3 for regression). 2 Best split search - for every candidate feature <i>x_j</i> in the sub-set, sort the <i>n</i> values once and evaluate candidate thresholds midway between successive sorted values; pick the pair (<i>x_j</i>, <i>θ</i>) that maximises impurity reduction (Gini, entropy, or variance). 3 Partitioning - send samples with <i>x_j</i> ≤ <i>θ</i> to the left child and the rest to the right. 4 Recurse until a stopping rule fires.
3. Stopping rules	min_samples_leaf – minimum samples allowed in a leaf (e.g. ≥ 1). max_depth – optional: restrict depth to curb over-fitting. min_impurity_decrease – halt if best split gives smaller gain than this threshold.
4. Leaf prediction	- Classification: majority class (optionally store class-probability vector). - Regression: mean response.
5. Aggregation of trees	- Classification: majority vote across the <i>T</i> trees (or average of class probabilities). - Regression: simple mean of predictions.

Main hyper-parameters to tune

Parameter	Effect
T (number of trees)	Larger T reduces variance but increases computation; usually 100–1000 is enough for convergence.
m_try	Controls correlation vs. strength of trees; lower m_try ⇒ more decorrelation (better global variance) but can weaken each tree.
min_samples_leaf / min_samples_split / max_depth	Regularisation knobs; increase them to fight over-fitting or noisy data.
Bootstrap size or use of subsampling without replacement	Alters bias/variance trade-off and runtime.

4 (b) Turning an axis-orthogonal RF into an oblique random forest

An **oblique** split is a linear decision surface

$$\mathbf{w}^\top \mathbf{x} \leq \theta,$$

using **multiple** variables simultaneously. To convert:

- At each node** draw the usual bootstrap subset of samples belonging to that node.
- Select a small random subset *m_try* of features** (or all features when dimensionality is low).
- Fit a fast two-class linear classifier** on that subset, treating the in-node samples’ class labels as the two classes:
 - choices: logistic regression, perceptron, linear SVM, LDA, or even a single-layer neural net without hidden units.
- Derive the hyper-plane parameters** (**w**, *θ*) from the classifier.
- Partition the samples** to left/right children using the sign of **w**[⊤]**x** − *θ*.
- Use the same stopping criteria** as in the orthogonal case.
- Keep bootstrap sampling, feature sub-sampling, and majority-vote aggregation exactly as before.

The result is an oblique decision tree ensemble that can slice the feature space at arbitrary angles, often requiring fewer levels to separate complex concepts.

4 (c) How to curb the computational cost of oblique forests

Strategy	Rationale
a. Use cheap linear models	Prefer perceptron with few iterations, ridge-regularised least-squares (a closed-form LDA), or incremental logistic regression instead of full QP SVM at every node.
b. Limit feature dimensionality per node	Randomly sample a small subset of features (<i>m_try</i> ≪ <i>p</i>), or perform feature hashing / PCA pre-projection; the linear solver’s cost then scales with <i>m_try</i> not <i>p</i> .
c. Early-stop the linear learner	Cap gradient-descent epochs or use a maximum wall-time per node; near-optimal hyper-planes are usually enough.
d. Depth-adaptive oblique splits	Employ oblique splits only in upper levels where node sizes are large; fall back to axis-orthogonal splits near leaves (small nodes) to avoid fitting many tiny linear models.
e. Batch re-use of hyper-planes	When several sibling nodes share identical feature subsets, reuse the same linear model after re-centering thresholds, or warm-start the solver with parent weights.
f. Parallel construction	Trees and even nodes within a tree are independent; distribute linear-model fitting across CPU cores or GPUs.
g. Regularisation shortcuts	Apply closed-form LDA (requires only covariance matrices) that can be updated incrementally, avoiding iterative optimisation.
h. Sparse weight enforcement	Add <i>L</i> ₁ (lasso) or group-lasso penalties so the learned w is sparse, cutting dot-product cost during prediction.

These measures together preserve most of the accuracy gain from oblique splits while keeping training and prediction times within practical bounds.